LANGGRAPH

Parallel Workflows

Video 6 -- Comprehensive Visual Notes

Detail	Value
Topic	Parallel Workflows -- Fan-Out / Fan-In Patterns
Core Problem	State conflicts when multiple nodes write simultaneously
Solutions	Partial Updates (different keys) + Reducers (same key)
Case Study	UPSC Essay Evaluator with 3 parallel LLM judges
Prerequisites	Video 5: Sequential Workflows, State, Edges
Key Imports	Annotated, operator.add, Pydantic BaseModel

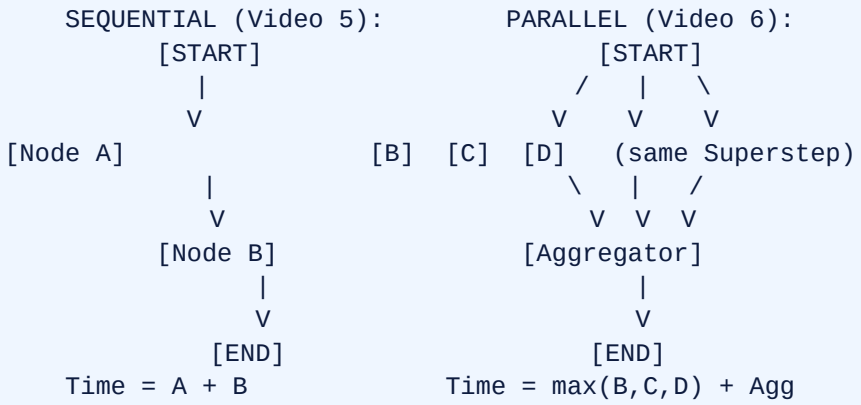
>> What You Will Learn

Fan-Out and Fan-In architecture for parallel execution. The critical InvalidUpdateError and how to avoid it. Two solutions: Partial Updates vs. Reducers (operator.add). Full UPSC Essay Evaluator case study. Extra: Pydantic structured output, async vs sync, decision guide, and code templates.

1. Core Concept: Sequential vs. Parallel

Unlike sequential workflows (A -> B -> C), **parallel workflows** execute multiple independent tasks **simultaneously**. This reduces total execution time and separates concerns.

Sequential vs. Parallel -- Side by Side



Analogy: Sequential = one cashier serving a queue. Parallel = three cashiers serving simultaneously. The queue clears 3x faster.

Aspect	Sequential (Video 5)	Parallel (Video 6)
Execution	One node at a time	Multiple nodes at once
Speed	Sum of all node times	Max of parallel nodes + aggregation
State Risk	None -- linear merge	Conflicts if same key updated
Complexity	Simple	Moderate (need reducers for shared keys)
Best For	Step-by-step pipelines	Multi-criteria evaluation, validation

2. Architecture Patterns: Fan-Out and Fan-In

FO

Fan-Out (One -> Many)

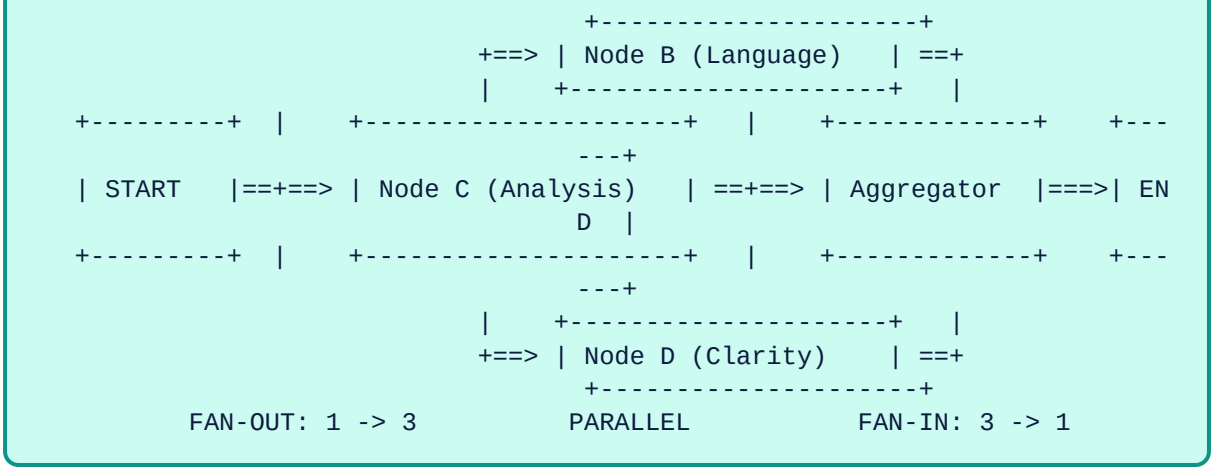
A single node connects to multiple downstream nodes. The graph engine triggers ALL connected nodes simultaneously in the same Superstep. Think of it as "broadcasting" the state.

FI

Fan-In (Many -> One)

Multiple parallel nodes all connect to a single downstream node (the Aggregator). The aggregator WAITS for ALL branches to finish before executing. It collects results.

Fan-Out + Fan-In -- The Complete Pattern



How to Wire Fan-Out / Fan-In in Code

```
graph = StateGraph(EssayState)

# Add all nodes
graph.add_node("eval_language", eval_language)
graph.add_node("eval_analysis", eval_analysis)
graph.add_node("eval_clarity", eval_clarity)
graph.add_node("aggregate", aggregate)

# FAN-OUT: START connects to ALL parallel nodes
graph.add_edge(START, "eval_language")
graph.add_edge(START, "eval_analysis")
graph.add_edge(START, "eval_clarity")

# FAN-IN: ALL parallel nodes connect to aggregator
graph.add_edge("eval_language", "aggregate")
graph.add_edge("eval_analysis", "aggregate")
graph.add_edge("eval_clarity", "aggregate")
graph.add_edge("aggregate", END)
```

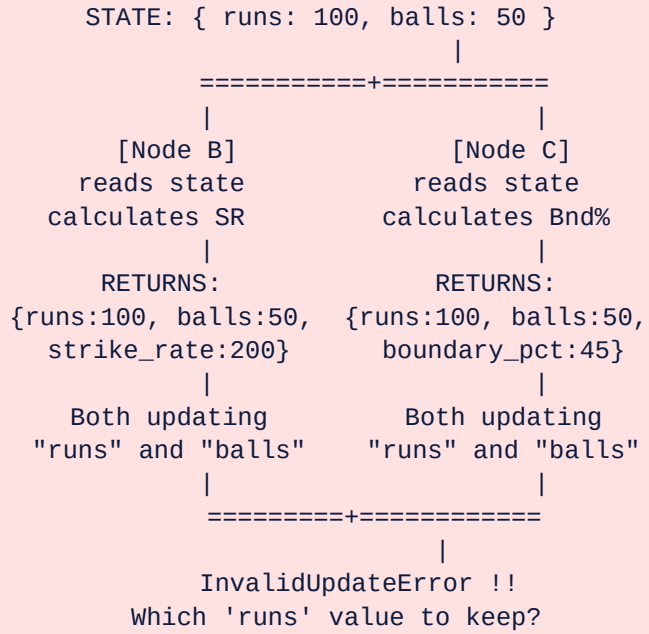
! Extra: Supersteps Revisited

Superstep 1: START runs, triggers fan-out. Superstep 2: Nodes B, C, D ALL execute simultaneously. Superstep 3: Aggregator runs ONLY after B, C, D have all finished. Superstep 4: END. The graph engine handles synchronization automatically.

3. The Critical Challenge: State Conflicts

This is the **most important concept** in this video. In sequential workflows, state updates happen one-at-a-time (safe). In parallel, multiple nodes read the SAME state and write back simultaneously -- creating conflicts.

The Problem: InvalidUpdateError



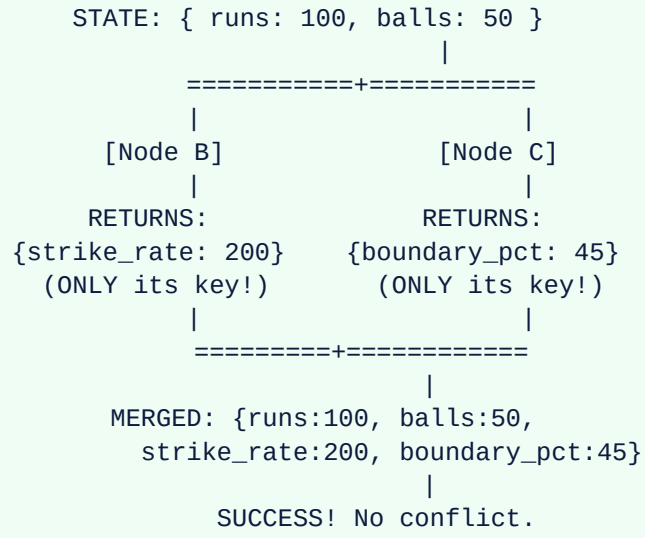
!! Why Does This Happen?

LangGraph's DEFAULT for state keys is OVERWRITE (replace old with new). When Node B returns {runs: 100, balls: 50, strike_rate: 200} and Node C returns {runs: 100, balls: 50, boundary_pct: 45}, both are writing 'runs' and 'balls' simultaneously. LangGraph cannot decide which version to keep. Analogy: Two people writing on the same line of a document at the same time.

4. Solution A: Partial State Updates (Different Keys)

When parallel nodes update **different** parts of the state, ensure each node **only returns the specific key it modified**. LangGraph merges distinct keys automatically.

Solution A: Each Node Returns ONLY Its Own Key



WRONG vs RIGHT

	What Node Returns	Result
WRONG	{"runs":100, "balls":50, "strike_rate":200}	InvalidUpdateError!
RIGHT	{"strike_rate": 200}	Merged cleanly
WRONG	{"runs":100, "balls":50, "boundary_pct":45}	InvalidUpdateError!
RIGHT	{"boundary_pct": 45}	Merged cleanly

!! Golden Rule for Parallel Nodes

Each parallel node should return ONLY the key(s) it is responsible for. Never echo back keys you read but didn't change. In sequential flows, extra keys are harmless (they just overwrite with the same value). In parallel, they cause fatal conflicts.

5. Solution B: Reducers (Same Key Updated by Multiple Nodes)

What if parallel nodes MUST update the **SAME key**? For example, three judges each adding their score to a shared list. You need a **Reducer** -- a function that tells LangGraph HOW to combine multiple updates.

Without vs. With Reducer

```
Three judges each return a score for 'scores' key:
      WITHOUT Reducer (default = overwrite):
               Judge 1: {scores: [8]}
               Judge 2: {scores: [7]}
               Judge 3: {scores: [9]}
      Final: scores = [9]    <-- LOST 8 and 7!
       WITH Reducer (operator.add = append):
               Judge 1: {scores: [8]}
               Judge 2: {scores: [7]}
               Judge 3: {scores: [9]}
   Final: scores = [8, 7, 9]   <-- ALL preserved!
```

Implementation: Annotated + operator.add

```
from typing import Annotated, TypedDict
import operator

class EssayState(TypedDict):
    essay: str

    # THE KEY LINE: operator.add as reducer
    # Means: CONCATENATE lists instead of overwriting
    individual_scores: Annotated[list[int], operator.add]

# Each judge returns a LIST (even for one score):
def judge_1(state) -> dict:
    return {"individual_scores": [8]}   # [8] appended

def judge_2(state) -> dict:
    return {"individual_scores": [7]}   # [7] appended

# LangGraph does: [] + [8] + [7] + [9] = [8, 7, 9]
```

!! CRITICAL: Why Wrap Score in a List?

Each judge returns {"individual_scores": [8]} NOT {"individual_scores": 8}. operator.add for LISTS does concatenation: [8] + [7] + [9] = [8, 7, 9]. For bare INTEGERS it does arithmetic: 8 + 7 + 9 = 24 (not what we want!). Always wrap values in a list when using operator.add as a list collector.

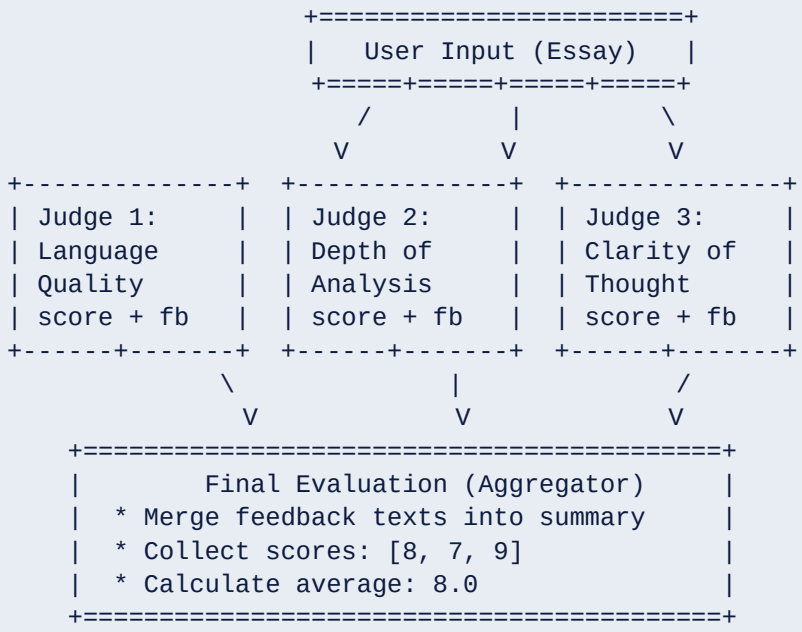
Extra: All Reducer Types

Reducer Syntax	Behavior	Example
Annotated[list, operator.add]	Concatenate lists	[8]+[7]+[9] = [8,7,9]
Annotated[int, operator.add]	Sum integers	8+7+9 = 24
Annotated[str, operator.add]	Concatenate strings	'a'+'b'+'c' = 'abc'
Annotated[list, add_messages]	Merge chat messages (dedup by ID)	Built-in for chat
Annotated[int, lambda a,b: max(a,b)]	Keep the maximum value	max(8,7,9) = 9
No Annotated (default)	Overwrite / replace	Only last value survives

6. Case Study: UPSC Essay Evaluator

The lecture builds a parallel evaluator where **3 LLM judges** assess an essay on different criteria simultaneously, then an aggregator combines their scores and feedback.

UPSC Essay Evaluator -- Full Architecture



State Design -- Both Solutions Working Together

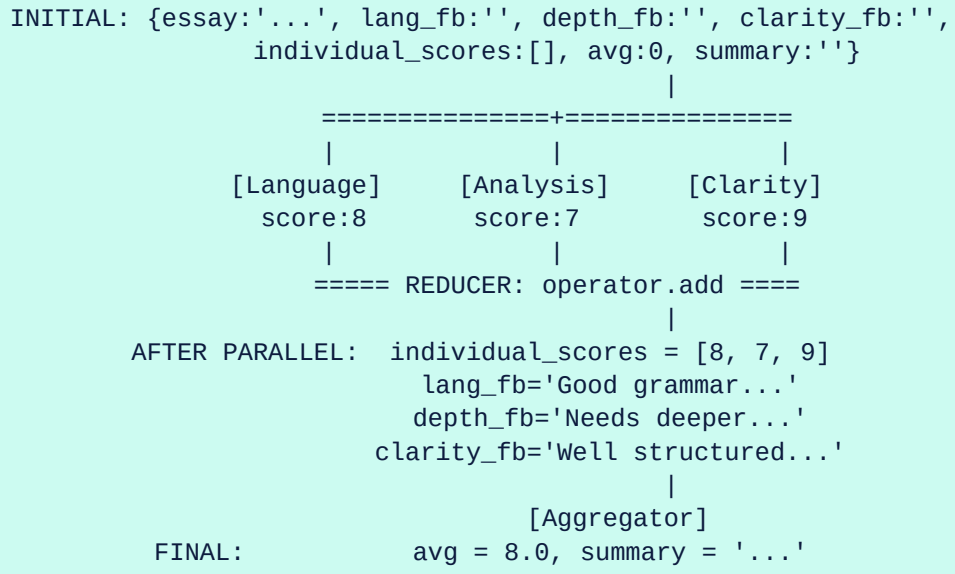
```
class EssayState(TypedDict):
    essay: str

    # SOLUTION A: Each judge writes to its OWN unique key
    language_feedback: str     # Only Judge 1 writes here
    depth_feedback: str        # Only Judge 2 writes here
    clarity_feedback: str      # Only Judge 3 writes here

    # SOLUTION B: All 3 judges write to the SAME key (reducer!)
    individual_scores: Annotated[list[int], operator.add]

    # Aggregator writes these
    average_score: float
    final_summary: str
```

State Evolution Through Pipeline



!! Both Solutions in One State!

Feedback keys (language_feedback, depth_feedback, clarity_feedback) use Solution A: each judge writes to its OWN unique key -- no conflict. Scores use Solution B: ALL three judges write to 'individual_scores' using operator.add reducer. You can freely mix both strategies in the same State definition.

7. Structured Output with Pydantic

When LLMs run in parallel, the aggregator needs predictable data. **Pydantic models** with `.with_structured_output()` guarantee each judge returns an exact schema (score as int, feedback as string).

```
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI

class JudgeOutput(BaseModel):
    score: int = Field(ge=0, le=10, description="Score 0-10")
    feedback: str = Field(description="Detailed feedback")

# Force LLM to return EXACTLY this schema
llm = ChatOpenAI(model='gpt-4o-mini')
structured_llm = llm.with_structured_output(JudgeOutput)

def eval_language(state: EssayState) -> dict:
    result = structured_llm.invoke(
        f"Evaluate language quality:\n{state['essay']}")
    # result.score is GUARANTEED to be int 0-10
    # result.feedback is GUARANTEED to be str
    return {
        "language_feedback": result.feedback,
        "individual_scores": [result.score],
    }
```

Without Pydantic	With Pydantic (.with_structured_output)
LLM might return: "Score is 8/10"	Returns: {"score": 8, "feedback": "..."}
Must parse/regex text manually	Automatic parsing and validation
No guarantee on data types	Type-safe (int, str, etc.)
No range validation	Pydantic enforces ge=0, le=10
Crashes if LLM changes format	Guaranteed schema compliance

!! Extra: Why Pydantic Matters Even More in Parallel

In sequential flows, one bad LLM output is caught immediately. In parallel, 3 LLMs run at once -- if ANY returns bad data, the aggregator crashes with cryptic errors. Pydantic validates output BEFORE it enters the state. It is input sanitization for your graph pipeline.

8. Extra: Sync vs. Async Execution

Aspect	Sync (Default)	Async
Execution model	Logically parallel, technically one-at-a-time	True concurrent I/O
Speed benefit	No real time savings for I/O	Real speedup for API calls
Node definition	def my_node(state):	async def my_node(state):
Invoke method	app.invoke(state)	await app.ainvoke(state)
3 judges at 2s each	~6 seconds total	~2 seconds total
When to use	Learning, prototyping	Production LLM pipelines

>> Practical Advice

For learning, sync is fine -- the LOGICAL parallelism (same Superstep) is the important concept. For production with 3+ LLM calls, use async. Each LLM API call spends most of its time waiting for network response (I/O-bound). Async lets Python fire all 3 calls, then collect results when they arrive.

9. Extra: Decision Guide -- Which Solution?

Decision Flowchart

```
Do parallel nodes write to the SAME key?
         |                        |
        NO                      YES
         |                        |
Use Solution A:          Use Solution B:
   PARTIAL UPDATES          REDUCERS
Each node returns        Annotated[list,
 only its own key         operator.add]
   You can MIX both in the same State!
  - Unique feedback keys --> Solution A
  - Shared scores list   --> Solution B
```

Scenario	Solution	Pattern
Nodes compute different metrics	A: Partial Updates	Return only your key
Nodes add items to shared list	B: Reducer	Annotated[list, operator.add]
Nodes produce scores to collect	B: Reducer	Return [score] not score
Mixed unique + shared keys	A + B Combined	Partial for unique, Reducer for shared

10. Extra: Common Pitfalls

Pitfall	What Goes Wrong	Fix
Returning full state	InvalidUpdateError	Return ONLY changed keys
No reducer for shared key	Last write wins (data lost)	Add Annotated[list, operator.add]
Bare int for list reducer	8+7+9=24 (arithmetic!)	Wrap in list: [score]
Unstructured LLM output	Aggregator parse crash	Use Pydantic .with_structured_output()
Missing fan-in edges	Aggregator runs too early	Connect ALL nodes to aggregator
Forgetting default values	KeyError in nodes	Pass all keys in invoke()

11. Quick Revision Checklist

Tag	Question

Fan-Out	How do you connect START to multiple parallel nodes in code?
Fan-In	What does the aggregator wait for before running?
Conflict	Why does returning full state cause InvalidUpdateError?
Solution A	When to use partial updates? What is the golden rule?
Solution B	What does Annotated[list, operator.add] do?
List Trap	Why return [score] instead of score for list reducer?
Pydantic	Why is structured output critical for parallel LLM nodes?
Async	When should you use async over sync for parallel graphs?
Case Study	Draw the UPSC Essay Evaluator graph from memory.

Cheat Sheet: Parallel Workflow Patterns

What	Code
Fan-Out	graph.add_edge(START, "node"); ...for each parallel node
Fan-In	graph.add_edge("node", "agg"); ...for each parallel node
List Reducer	key: Annotated[list[int], operator.add]
Safe Partial Return	return {"only_my_key": value}
Structured LLM	llm.with_structured_output(PydanticModel)
Async Invoke	await app.ainvoke(initial_state)
Pydantic Model	class Out(BaseModel): score: int = Field(ge=0, le=10)

What's Next?

Next video covers **Conditional Edges and Routing** -- adding if/else decision logic to your graphs. Combined with sequential (V5) and parallel (V6), routing gives you the building blocks for truly intelligent agents. Make sure you can explain InvalidUpdateError and both solutions from memory!